

Pull Request Dokumentation

Stand: 22.05.2026

Repository URL: https://github.com/FionnLaesser/M324_PROJEKT_TODOLIST

Was ist ein Pull Request?

Ein Pull Request ist eine Anfrage, Änderungen aus einem Branch in einen anderen Branch zu übernehmen. In unserem Projekt wird dafür ein Feature-Branch verwendet, der später in den **main**-Branch gemerged wird.

Bei GitLab heisst das gleiche Prinzip meistens Merge Request. Die Bedeutung ist gleich: Änderungen werden zuerst separat vorbereitet und danach überprüft, bevor sie in den Hauptstand des Projekts übernommen werden.

Warum verwendet man Pull Requests?

Pull Requests helfen dabei, Änderungen kontrolliert in ein Projekt zu übernehmen. Sie sind besonders nützlich, wenn mehrere Personen am gleichen Projekt arbeiten.

- Änderungen werden überprüft, bevor sie in **main** kommen.
- Andere Personen können Feedback geben.
- Fehler werden früher erkannt.
- Die Code-Qualität wird verbessert.
- Der **main**-Branch bleibt stabiler.
- Der Ablauf bleibt nachvollziehbar, weil Commits, Kommentare und Reviews im Pull Request sichtbar sind.

Vorbereitung

Vor dem Erstellen eines Pull Requests wird zuerst ein eigener Branch erstellt. Dieser Branch enthält die Änderungen, die später in **main** übernommen werden sollen.

Wichtige Git-Befehle:

```
git checkout main
git pull origin main
git checkout -b feature/todo-validation
git status
git add .
git commit -m "Verhindere leere Todos"
git push origin feature/todo-validation
```

In unserem Projekt wurde für die echte Änderung der Branch **feature/todo-validation** verwendet. Die Änderungen wurden dort vorbereitet und anschliessend auf GitHub hochgeladen.

Echte Änderung im Projekt

Für die notenrelevante Pull-Request-Bewertung wurde eine kleine echte Änderung an der Todo-Liste umgesetzt.

Problem:

Leere Todos sollen nicht erstellt werden. Das gilt auch dann, wenn im Eingabefeld nur Leerzeichen stehen.

Umsetzung:

- Im Frontend wird der Button **Absenden** deaktiviert, solange die Eingabe leer ist.
- Beim Absenden wird weiterhin geprüft, ob der Text nach dem Entfernen von Leerzeichen leer ist.
- Im Backend werden leere oder nur aus Leerzeichen bestehende Todo-Beschreibungen ignoriert.
- Ein kleiner Test prüft, dass ein Todo mit leerer Beschreibung nicht gespeichert wird.

Geänderte Dateien:

Datei	Änderung
<code>frontend/src/App.jsx</code>	Deaktiviert den Absenden-Button bei leerer Eingabe.
<code>backend/src/main/java/com/example/demo/DemoApplication.java</code>	Verhindert leere Todo-Einträge auch direkt über die API.
<code>backend/src/test/java/com/example/demo/DemoApplicationTests.java</code>	Ergänzt einen Test für leere Todo-Beschreibungen.

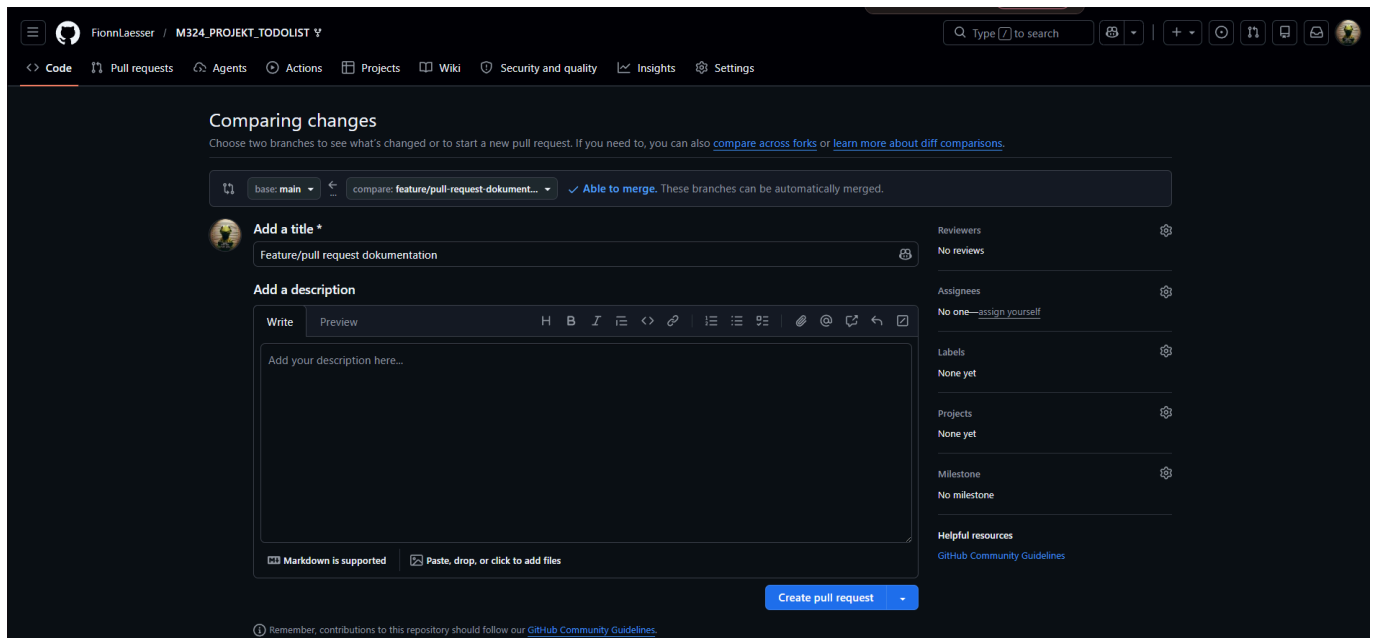
Pull Request erstellen

Nach dem Push des Feature-Branche kann auf GitHub ein Pull Request erstellt werden. Dabei wird ausgewählt, von welchem Branch in welchen Zielbranch gemerged werden soll.

Für dieses Projekt war die Auswahl:

- Base-Branch: `main`
- Compare-Branch: `feature/todo-validation`

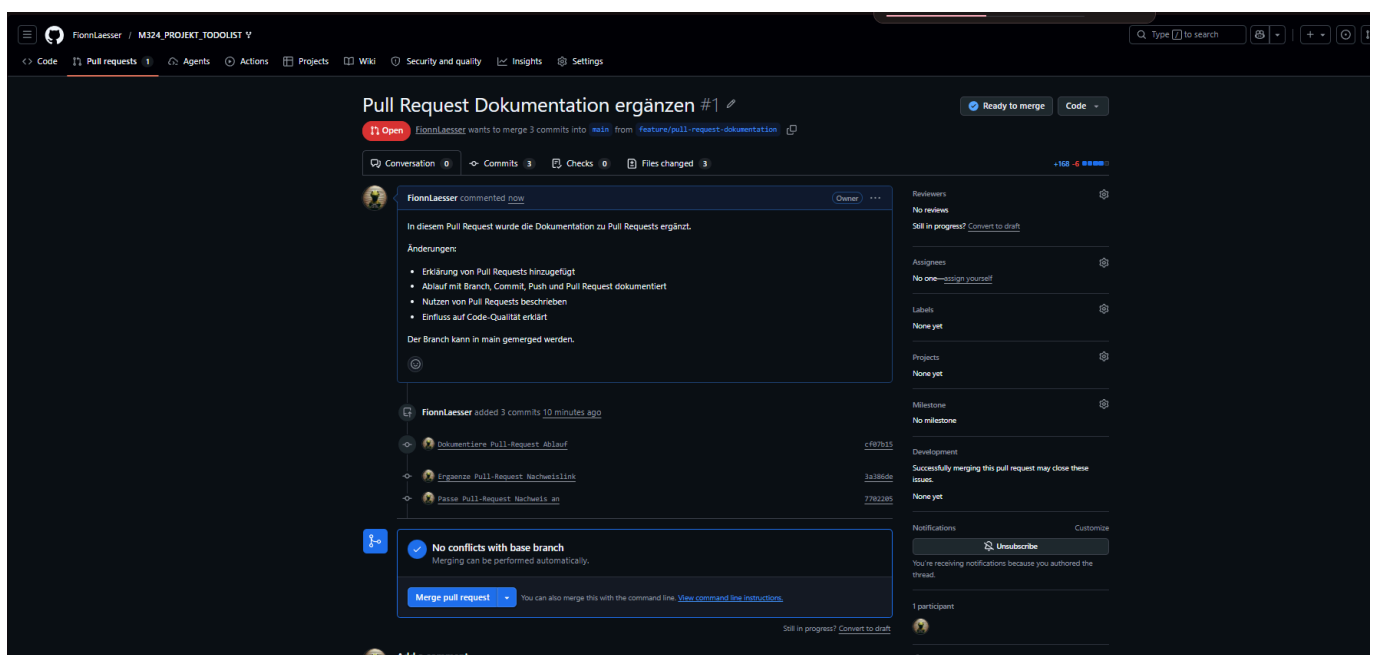
Danach wurden ein Titel und eine Beschreibung ergänzt. Die Beschreibung sollte kurz erklären, was geändert wurde und warum diese Änderung gemacht wurde.



Pull Request überprüfen

Nach dem Erstellen des Pull Requests zeigt GitHub an, ob die Änderungen gemerged werden können. Wichtig ist, dass keine Merge-Konflikte vorhanden sind.

In unserem Fall zeigte GitHub den Status **Ready to merge**. Ausserdem wurde angezeigt, dass keine Konflikte mit dem Base-Branch vorhanden waren. Damit war der Pull Request technisch bereit für den Merge.



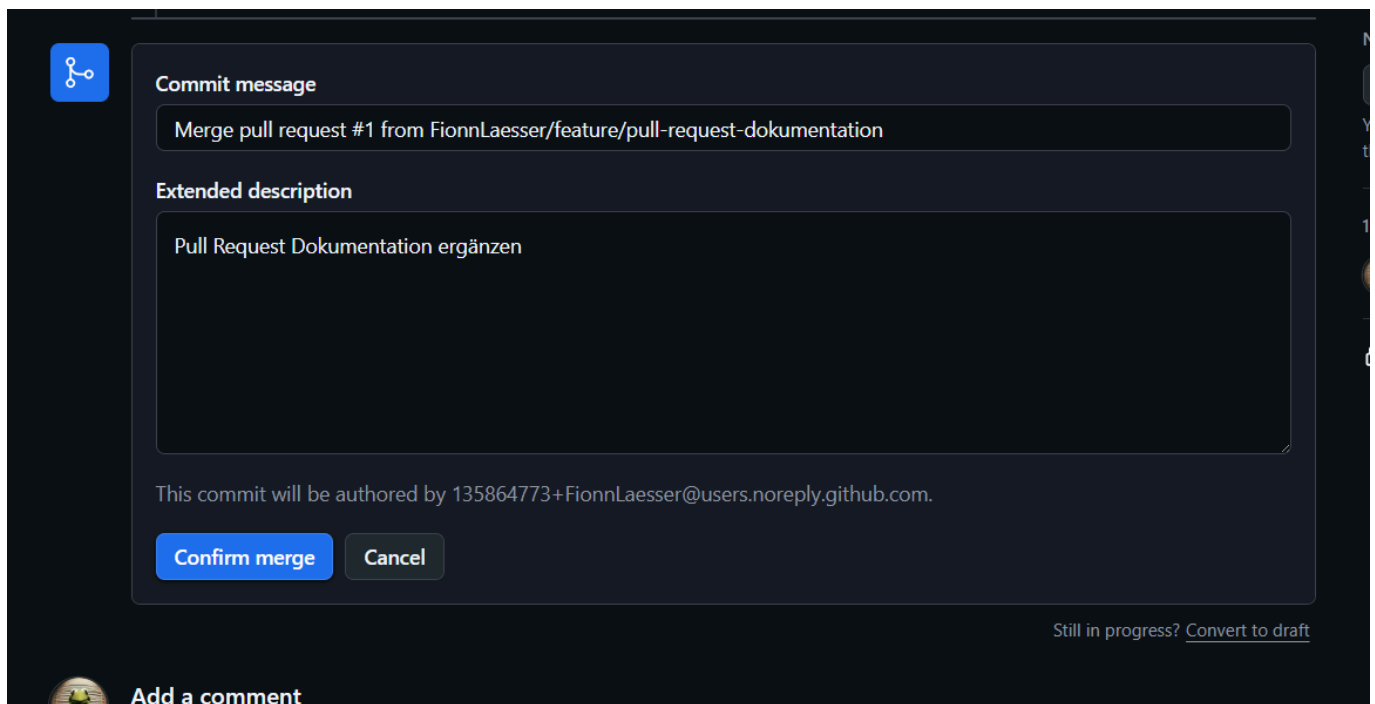
Pull Request mergen

Wenn der Pull Request überprüft wurde und keine Konflikte vorhanden sind, kann er in den **main**-Branch gemerged werden. Dafür wird auf GitHub zuerst die Schaltfläche **Merge pull request** verwendet.

Anschliessend muss der Merge nochmals bestätigt werden. Dieser Schritt verhindert, dass Änderungen versehentlich in **main** übernommen werden.



Nach der Bestätigung zeigt GitHub an, dass der Pull Request erfolgreich gemerged wurde. Die Änderungen sind damit im **main**-Branch enthalten.



Nachweise mit Screenshots

Die folgenden Screenshots dokumentieren den Ablauf des Pull Requests.

Datei	Zweck	Status
01_pr_problem_keine_rechte.png	Zeigt, dass zuerst kein Pull Request ins WISS-GB Repository erstellt werden konnte, weil nur Collaborators berechtigt waren.	TODO: Screenshot 01_pr_problem_keine_rechte.png einfügen
02_pr_erstellen_formular.png	Zeigt das Formular zum Erstellen des Pull Requests mit Titel und Beschreibung.	Vorhanden
03_pr_offen_ready_to_merge.png	Zeigt den geöffneten Pull Request mit Status ready to merge und ohne Konflikte.	Vorhanden
04_pr_confirm_merge.png	Zeigt den Schritt Confirm merge.	Vorhanden
05_pr_erfolgreich_gemerged.png	Zeigt, dass der Pull Request erfolgreich in <code>main</code> gemerged wurde.	Vorhanden

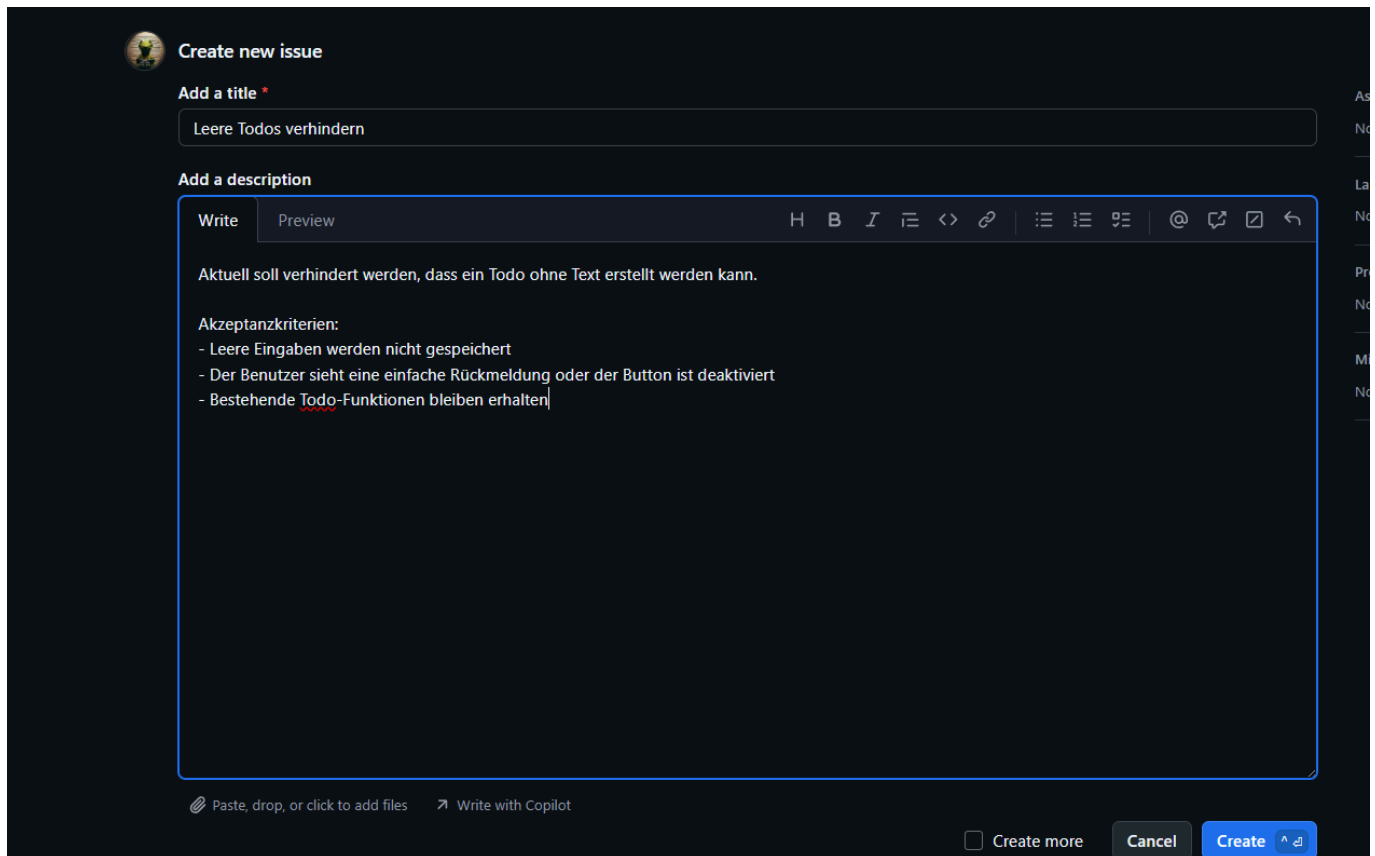
TODO: Screenshot `01_pr_problem_keine_rechte.png` einfügen, sobald der passende Screenshot vorhanden ist.

Bewertungspunkte zur Todo-Validierung

Die folgenden Abschnitte dokumentieren die notenrelevante Pull-Request-Arbeit zur Todo-Validierung. Es werden nur Bilder verwendet, die einen echten Arbeitsschritt zeigen. Es wird kein Screenshot verwendet, der die Meldung `There isn't anything to compare` enthält.

1. Issue erstellt

Für die Änderung wurde ein Issue erstellt. Das Issue beschreibt, dass leere Todos verhindert werden sollen und nennt die wichtigsten Akzeptanzkriterien.



Create new issue

Add a title *

Leere Todos verhindern

Add a description

Write Preview

Aktuell soll verhindert werden, dass ein Todo ohne Text erstellt werden kann.

Akzeptanzkriterien:

- Leere Eingaben werden nicht gespeichert
- Der Benutzer sieht eine einfache Rückmeldung oder der Button ist deaktiviert
- Bestehende Todo-Funktionen bleiben erhalten

Paste, drop, or click to add files Write with Copilot

Create more Cancel Create

2. Feature-Branch erstellt

Für die Umsetzung wurde ein eigener Branch erstellt. Der Branch heisst `feature/todo-validation` und trennt die Änderung vom `main`-Branch.

```
PS C:\Users\flaes\Desktop\Schule\Aktuell\324\Projekt> git checkout main
>> git pull origin main
>> git checkout -b feature/todo-validation
Already on 'main'
Your branch is up to date with 'origin/main'.
From github.com:FionnLaesser/M324_PROJEKT_TODO_LIST
* branch          main          -> FETCH_HEAD
Already up to date.
Switched to a new branch 'feature/todo-validation'
PS C:\Users\flaes\Desktop\Schule\Aktuell\324\Projekt>
```

3. Änderungen implementiert

Die Code-Änderung verhindert, dass leere Todo-Beschreibungen gespeichert werden. Dabei wird geprüft, ob die Beschreibung `null`, leer oder nur aus Leerzeichen besteht.

```
diff --git a/backend/src/main/java/com/example/demo/DemoApplication.java b/backend/src/main/java/com/example/demo/DemoApplication.java
index 240cae2..b709530 100644
--- a/backend/src/main/java/com/example/demo/DemoApplication.java
+++ b/backend/src/main/java/com/example/demo/DemoApplication.java
@@ -139,6 +139,11 @@ public class DemoApplication {
    try {
        Task task;
        task = mapper.readValue(taskdescription, Task.class);
+       if (task.getTaskdescription() == null || task.getTaskdescription().isBlank()) {
+           System.out.println(">>>empty task descriptions are ignored!");
+           return "redirect:/";
+       }
        task.setTaskdescription(task.getTaskdescription().trim());
        if (task.getPriority() == null || task.getPriority().isBlank()) {
            task.setPriority("Mittel");
        }
    }
}
diff --git a/backend/src/test/java/com/example/demo/DemoApplicationTests.java b/backend/src/test/java/com/example/demo/DemoApplicationTests.java
index fb60d89..f6a2443 100644
--- a/backend/src/test/java/com/example/demo/DemoApplicationTests.java
+++ b/backend/src/test/java/com/example/demo/DemoApplicationTests.java
@@ -13,4 +13,13 @@ class DemoApplicationTests {
    // ...
}
```

4. Commit mit closes #2

Der Commit enthält den Bezug zum Issue mit **closes #2**. Dadurch kann GitHub das Issue nach dem Merge automatisch schliessen.

```
>> git diff
>> git add .
>> git commit -m "Verhindere leere Todos closes #2"
>> git push origin feature/todo-validation
```

5. Pull Request erstellt

Der Pull Request wurde von **feature/todo-validation** nach **main** erstellt. Der Screenshot zeigt, dass GitHub echte Unterschiede erkennt und der Branch gemerged werden kann.

Comparing changes
Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: **main** ← compare: **feature/todo-validation** ✓ **Able to merge**. These branches can be automatically merged.

Add a title *
Leere Todos verhindern

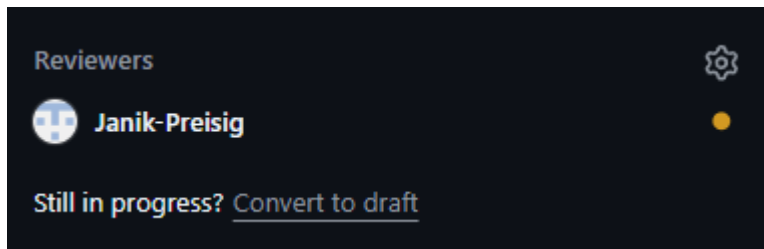
Add a description
Write | Preview
Dieser Pull Request verhindert, dass leere Todos erstellt werden können.
Closes #2
Änderungen:
- Leere Eingaben werden abgefangen
- Todo-Erstellung bleibt für gültige Eingaben gleich
- Dokumentation wurde ergänzt

Reviewers: No reviews
Assignees: No one—assign yourself
Labels: None yet
Projects: None yet
Milestone: No milestone
Development: Use [closing keywords](#) in the description to automatically close issues

Create pull request

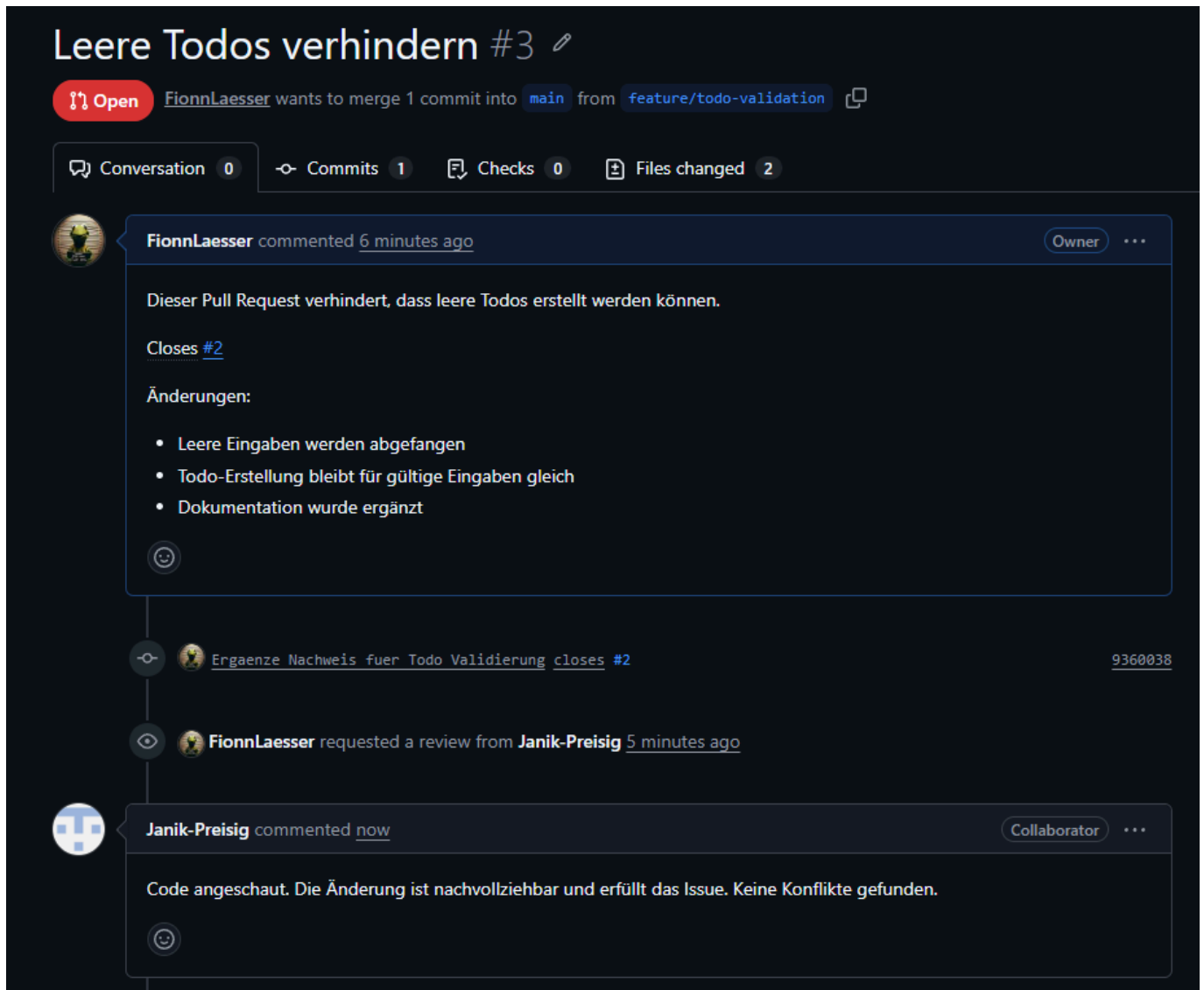
6. Sparring als Reviewer

Für das Review wurde eine andere Person als Reviewer eingetragen. Dadurch wird sichtbar, dass die Änderung nicht nur allein umgesetzt, sondern auch im Team angeschaut wurde.



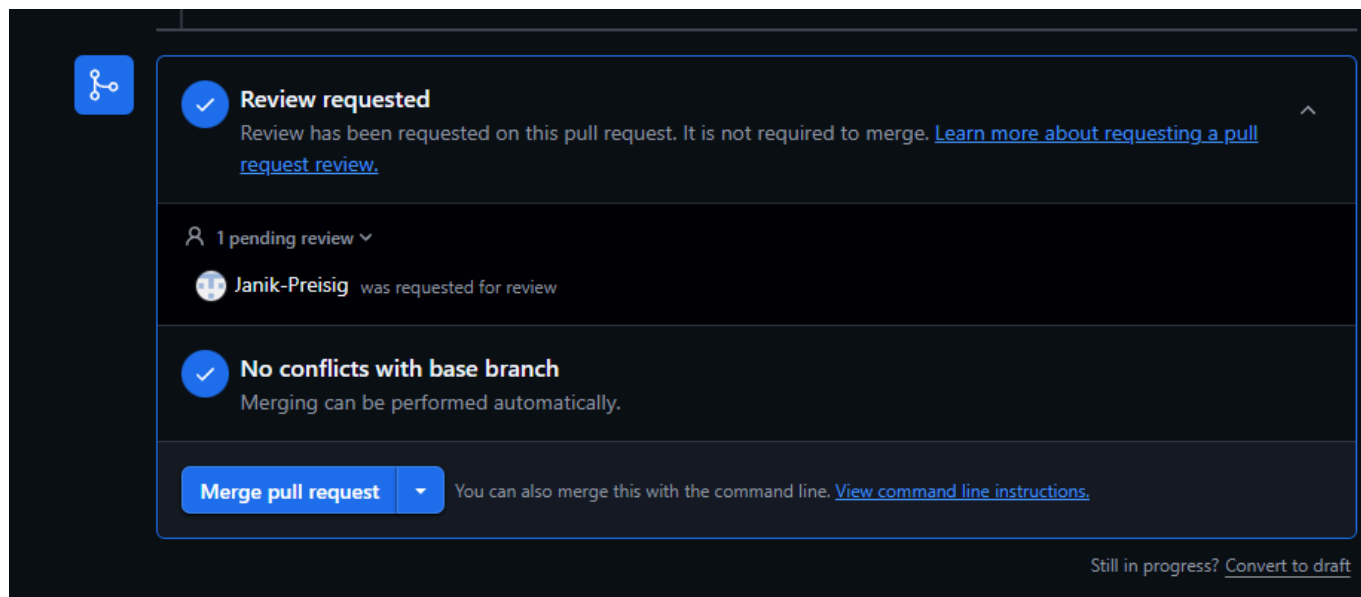
7. Review dokumentiert

Das Review wurde im Pull Request dokumentiert. Der Kommentar bestätigt, dass die Änderung nachvollziehbar ist, das Issue erfüllt und keine Konflikte vorhanden sind.



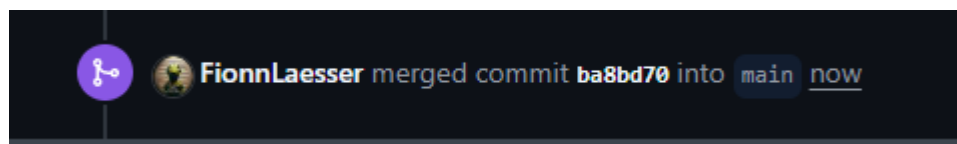
8. Pull Request genehmigt und durchgeführt

Der Pull Request war bereit zum Merge. GitHub zeigte keine Konflikte mit dem Base-Branch an. Damit konnte der Pull Request durchgeführt werden.



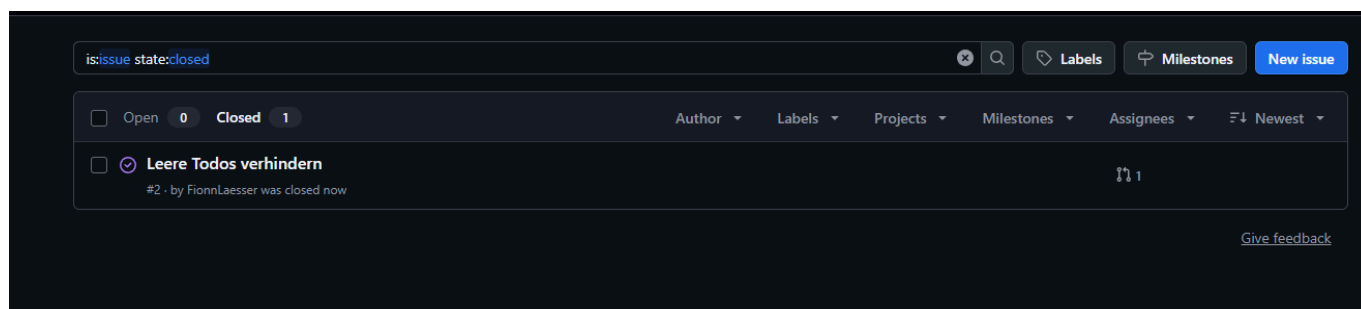
9. Pull Request online gemerged

Der Pull Request wurde online auf GitHub in den **main**-Branch gemerged. Damit wurden die Änderungen aus dem Feature-Branch in den Hauptstand übernommen.



10. Issue automatisch geschlossen

Nach dem Merge wurde das Issue automatisch geschlossen, weil der Commit oder Pull Request den Hinweis **closes #2** enthielt.



11. Lokales Repo aktualisiert

Nach dem Online-Merge wurde lokal **main** aktualisiert. Der vorhandene Screenshot zeigt den Pull-Vorgang und den dabei sichtbaren lokalen Konfliktstand. Dieser Konflikt muss lokal bereinigt werden, bevor der Arbeitsstand vollständig sauber ist.

```
Already on 'main'
Your branch and 'origin/main' have diverged,
and have 1 and 2 different commits each, respectively.
(use "git pull" if you want to integrate the remote branch with yours)
From github.com:FionnLaesser/M324_PROJEKT_TODOLIST
* branch          main          -> FETCH_HEAD
Auto-merging backend/src/test/java/com/example/demo/DemoApplicationTests.java
CONFLICT (content): Merge conflict in backend/src/test/java/com/example/demo/DemoApplicationTests.java
Automatic merge failed; fix conflicts and then commit the result.
32d39ef (HEAD -> main, backup-main-vor-fix) Verhindere leere Todos closes #2
a05444d Dokumentiere Pull-Request Ablauf
a97c02d Merge pull request #1 from FionnLaesser/feature/pull-request-dokumentation
7702205 Passe Pull-Request Nachweis an
3a386de Ergaenze Pull-Request Nachweislink
```

Nutzen von Pull Requests

Pull Requests erleichtern die Zusammenarbeit, weil Änderungen nicht direkt in den **main**-Branch geschrieben werden. Stattdessen werden sie zuerst in einem eigenen Branch gesammelt und danach überprüft.

Ein grosser Vorteil ist, dass andere Personen den Code anschauen können, bevor er übernommen wird. Dadurch können Fehler, unklare Stellen oder fehlende Tests früher gefunden werden. Auch die Beschreibung im Pull Request hilft, die Änderung später nachzuvollziehen.

Pull Requests können aber auch zusätzlichen Aufwand verursachen. Für sehr kleine Änderungen dauert der Ablauf manchmal länger als ein direkter Commit. Ausserdem kann ein Pull Request blockiert werden, wenn Reviews fehlen oder wenn es Merge-Konflikte gibt.

Trotzdem ist der Nutzen für ein Teamprojekt grösser als der Aufwand. Pull Requests verbessern die Code-Qualität, machen Änderungen transparenter und schützen den **main**-Branch vor fehlerhaften oder unvollständigen Änderungen.

Fazit

In diesem Projekt wurde ein Feature-Branch erstellt, die Dokumentation zu Pull Requests ergänzt und der Branch über einen Pull Request in **main** gemerged. Die Screenshots zeigen die wichtigsten Schritte: Pull Request erstellen, Status prüfen, Merge bestätigen und erfolgreichen Merge kontrollieren.

Quellen

- GitHub Docs: About pull requests, <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>
- GitHub Docs: Creating a pull request, <https://docs.github.com/en/articles/creating-a-pull-request>
- GitHub Docs: Merging a pull request, <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/incorporating-changes-from-a-pull-request/merging-a-pull-request>